第二章 高级语言及其文法

重点：文法的定义与分类，CFG的语法树及二义性、程序设计语言的定义。

难点：程序设计语言的语义定义。

2.3 文法的定义

如何实现语言结构的形式化描述?

考虑赋值语句的形式:

左部量 = 右部表达式

a = a+a

b = m[3]+b

m[1] = a+m[2]

句子的组成规则

- <赋值语句>→<左部量>=<右部表达式>
- <左部量>→<简单变量>
- <左部量>→ <下标变量>
- <简单变量>→a
- <简单变量>→b
- <简单变量>→ c
- <下标变量>→m[1]
- <下标变量>→m[2]
- <下标变量>→m[3]
- <右部表达式>→<简单变量><运算符><简单变量>
- <右部表达式>→<简单变量><运算符><下标变量>
- <右部表达式>→<下标变量><运算符><简单变量>
- <右部表达式>→<下标变量><运算符><下标变量>
- <运算符>→ +
- <运算符>→ -

问题：如何用符号来描述？即如何形式化？

定义句子的规则的语法组成

——终结符号集，非终结符号集，语法规则，开始符号

非终结符号集V=

{<赋值语句>，<左部量>，<右部表达式>，<简单变量>，<下标变量>，<运算符>}

终结符号集T=

{a , b, c, m[1], m[2], m[3], +, -}

语法规则集P =

{<赋值语句>→<左部量>=<右部表达式>， ……}

开始符号S = <赋值语句>

文法G的形式定义

定义2.16 文法G为一个四元组:

$$G=(V,\ T,\ P,\ S)$$

- V：非终结符(Variable)集
  - 语法变量（成分）——代表某个语言的各种子结构
- T：终结符(Terminal)集
  - 语言的句子中出现的字符，$V\cap T=\varnothing$
- S：开始符号(Start Symbol)，$S\in V$
  - 代表文法所定义的语言，至少在产生式左侧出现一次

文法G的形式定义

- P：产生式(Product)集合

$\alpha \rightarrow \beta$，被称为产生式（定义式），读作：α定义为β。其中$\alpha \in (V \cup T)^+$，且α中至少有 V中元素的一个出现。$\beta \in (V \cup T)^*$。α称为产生式$\alpha \rightarrow \beta$的左部(Left Part)，β称为产生式$\alpha \rightarrow \beta$的右部(Right Part)。

产生式定义各个语法成分的结构（组成规则）

例 赋值语句的文法

- *V*={<赋值语句>，<左部量>，<右部表达式>，<简单变量>，　<下标变量>，<运算符>}
- *T*={a，b，c，m[1]，m[2]，m[3]，+，-}
- *P*={<赋值语句>→<左部量>=<右部表达式>，

  <左部量>→<简单变量>, <左部量>→ <下标变量>,

  <简单变量>→a, <简单变量>→b, <简单变量>→ c,

  <下标变量>→m[1], <下标变量>→m[2]，

  <下标变量>→m[3],

  <右部表达式>→<简单变量> <运算符><简单变量>,

  <右部表达式>→<简单变量><运算符> <下标变量>,

  <右部表达式>→<下标变量><运算符><简单变量>，

  <右部表达式>→<下标变量><运算符><下标变量>，

  <运算符>→ +，<运算符>→ -}
- *S*=<赋值语句>

例 赋值语句的文法（续）

- 符号化之后：
- G=({A，B，E，C，D，P}，{a，b，c，m[1]，m[2]，m[3]，+，-}，P，A)，

  其中，P={$A\rightarrow B=E$，$B\rightarrow C$，$B\rightarrow D$，$C\rightarrow$a，$C\rightarrow$b，$C\rightarrow$ c，$D\rightarrow$m[1]，$D\rightarrow$m[2]，$D\rightarrow$m[3]，$E\rightarrow COC$，$E\rightarrow COD$，$E\rightarrow DOC$，$E\rightarrow DOD$，$O\rightarrow$ +，$O\rightarrow$ -}

产生式的简写

- 对一组有相同左部的产生式

$$\alpha \rightarrow \beta_1,\ \alpha \rightarrow \beta_2 \ldots,\ \alpha \rightarrow \beta_n$$

可以简单地记为：

$$\alpha \rightarrow \beta_1 | \beta_2 | \ldots | \beta_n$$

读作：α定义为或者β_1，或者β_2，…，或者β_n。并且称它们为α产生式。β_1，β_2，…，β_n称为候选式(Candidate)

- 对一个文法，只列出该文法的所有产生式，且所列的第一个产生式的左部是该文法的开始符号。

问题：有了定义句子的规则，如何判定某一句子是否属于某语言？

句子的派生(推导)——从产生语言的角度

<赋值语句>

⇒ <左部量> = <右部表达式>

⇒ <简单变量> = <右部表达式>

⇒ a = <右部表达式>

⇒ a = <简单变量><运算符><简单变量>

⇒ a = a <运算符><简单变量>

⇒ a = a + <简单变量>

⇒ a = a + a

句子的识别(归约)——从识别语言的角度

a = a + a

⇐ a = a + <简单变量>

⇐ a = a <运算符><简单变量>

⇐ a = <简单变量><运算符><简单变量>

⇐ a = <右部表达式>

⇐ <简单变量> = <右部表达式>

⇐ <左部量> = <右部表达式>

⇐ <赋值语句>

- 派生与识别均根据规则

基于产生式的变换——推导或归约

- 定义2.17 设$G=(V，T，P，S)$是一个文法，如果$\alpha\to\beta\in P$，γ，$\delta\in(V\cup T)^*$，则称$\gamma\alpha\delta$在G中直接推导出$\gamma\beta\delta$，记作：

$$\gamma\alpha\delta \underset{G}{\Rightarrow} \gamma\beta\delta$$

- 读作：$\gamma\alpha\delta$在文法G中直接推导出$\gamma\beta\delta$。在不特别强调推导的直接性时，“直接推导”可以简称为推导(derivation)，有时我们也称推导为派生。

- 与之相对应，也可以称$\gamma\beta\delta$在文法G中直接归约成$\gamma\alpha\delta$。在不特别强调归约的直接性时，“直接归约”可以简称为归约(reduction)。由于这里实际是将$\gamma\beta\delta$中的β变成了α，而γ和δ都没有变化，所以又称将β归约成α。

(多步)推导

- $\alpha_0 \Rightarrow \alpha_1 \Rightarrow \alpha_2 \Rightarrow \ldots \Rightarrow \alpha_n$
  - 记为 $\alpha_0 \overset{n}{\Rightarrow} \alpha_n$ (恰用n步)
  - $\alpha_0 \overset{+}{\Rightarrow} \alpha_n$ （至少一步）
  - $\alpha_0 \overset{*}{\Rightarrow} \alpha_n$ （若干步:零步或多步）

文法G产生的语言

- 定义2.19 设有文法$G=(V，T，P，S)$，则称

$$L(G)=\{w \mid S \overset{+}{\Rightarrow} w \ \& \ w \in T^* \}$$

为文法G产生的语言(language)。$\forall w \in L(G)$，w称为G产生的一个句子(sentence)。

显然，对于任意一个文法G，G产生的语言$L(G)$就是该文法的开始符号S所对应的集合$L(S)$。

文法G产生的语言(续)

$$L(G)=\{x \mid S \Rightarrow^{*} x \text{ and } x \in T^{*}\}$$

- 文法 *G*可以派生出多少个句子?
- 文法 *G*的作用——语言的有穷描述
  - 以有限的规则描述无限的语言现象

- 有限:
  - 产生式集合、终结符集合、非终结符集合
- 无限:
  - 可以导出无穷多个句子（*L*也可是有穷）

推导/归约举例

$A \Rightarrow B=E$　　有产生式$A \rightarrow B=E$，所以可以将A换成$B=E$

$\Rightarrow C=E$　　有产生式$B \rightarrow C$，所以可以将$B=E$中的B换成C

$\Rightarrow a=E$　　有产生式$C \rightarrow a$，所以可以将$C=E$中的C换成a

$\Rightarrow a=COD$　　有产生式$E \rightarrow COD$，所以可以将a=E中的E换成COD

$\Rightarrow a=bOD$　　有产生式$C \rightarrow b$，所以可以将a=COD中的C换成b

$\Rightarrow a=b+D$　　有产生式$O \rightarrow +$，所以可以将a=bOD的O换成+

$\Rightarrow a=b+m[1]$　有产生式$D \rightarrow m[1]$，所以可以将a=b+D的D换成m[1]

A→B=E
B→C| D
C→a | b | c
D→m[1] | m[2]| m[3]
E→COC | COD | DOC | DOD
O→ + | -

句型与句子

- 定义2.20 设文法$G=(V，T，P，S)$，对于$\forall\alpha\in(V\cup T)^*$，如果$S\overset{*}{\Rightarrow}\alpha$，则称$\alpha$是$G$产生的一个句型(sentential form)，简称为句型

- 对于任意文法$G=(V，T，P，S)$，G产生的句子和句型的区别在于句子满足$w\in T^*$，而句型满足$\alpha\in(V\cup T)^*$

句型与句子

- 句子w是从S开始，在G中可以推导出来的终结符号行，它不含语法变量；

- 句型α是从S开始，在G中可以推导出来的符号行，它可能含有语法变量；

- 句子一定是句型；但句型不一定是句子

单选题 20分

设置

文法G产生的( )的全体是该文法描述的语言。

A 终结符集

B 非终结符集

C 句子

D 句型

给定文法A→bA|cc，则以下符号串是该文法句子的是( )。

A cc

B bcbc

C bbbccc

D bccbcc

E bbbbcc

单选题 20分

设置

文法 S→aaS|abc 定义的语言是( )。

A $\{a^{2k-1}bc|k>0\}$

B $\{a^k a^k bc|k>0\}$

C $\{a^k bc|k>0\}$

D $\{a^{2k}bc|k>0\}$

2.4 文法的分类(Chomsky体系)

- 根据语言结构的复杂程度（形式语言）
  - 涉及文法的复杂程度、分析方法的选择
  - 反映文法描述语言的能力

- 如果*G*满足文法定义的要求，则 *G*是 0 型文法（短语结构文法PSG: Phrase Structure Grammar ）。

- *L*(*G*)为PSL。

（1）上下文有关文法(CSG)

- 如果对于$\forall \alpha \to \beta \in P$，均有$|\beta| \geq |\alpha|$成立（$S \to \varepsilon$除外），则称$G$为1型文法
  - 即：上下文有关文法（CSG——Context Sensitive Grammar）

- $L(G)$为1型/上下文有关/敏感语言(CSL)

(2) 上下文无关文法(CFG)

- 如果对于$\forall \alpha \to \beta \in P$，均有$|\beta| \geq |\alpha|$，并且$\alpha \in V$成立，则称$G$为 2 型文法
  - 即：上下文无关文法(CFG: Context Free Grammar)

- $L(G)$为2型/上下文无关语言（CFL）
  - CFG能描述程序设计语言的多数语法成分

(2) 正规文法(RG)

- 设A、$B \in V$，$a \in T$或为ε

  右线性(Right Linear)文法：$A \rightarrow aB$或$A \rightarrow a$

  左线性(Left Linear)文法：$A \rightarrow Ba$或$A \rightarrow a$

  都是 3 型文法(正规文法 Regular Grammar -RG)

- $L(G)$为3型/正规集/正则集/正则语言（RL）
  - 能描述程序设计语言的多数单词
  - 左、右线性文法不可混用

例 标识符的文法

- $S \rightarrow L|LT$

  $T \rightarrow L|N|TL|TN$

  $L \rightarrow a|b|c|d$

  $N \rightarrow 0|1|2|3|4|5$

- $S \rightarrow a|b|c|d$

  $S \rightarrow aT|bT|cT|dT$

  $T \rightarrow a|b|c|d|0|1|2|3|4|5$

  $T \rightarrow aT|bT|cT|dT|0T$

  $T \rightarrow 1T|2T|3T|4T|5T$

非上下文无关的语言结构

- 程序设计语言的有些语言结构不能用上下文无关文法描述
- 例

$$L_1=\{wcw|w\in\{a,b\}^+\}$$

$aabcaab$就是L_1的一个句子

这个语言是检查程序中标识符的声明应先于引用的抽象

非上下文无关的语言结构

- 例

$$L_2=\{a^n b^m c^n d^m | n,m\geq 0\}$$

- 它是检查过程声明的形参个数和过程引用的参数个数是否一致问题的抽象

Chomsky体系——总结

$G=(V, T, P, S)$是一个文法,$\alpha \rightarrow \beta \in P$

* G是0型文法，$L(G)$是0型语言；

  ---其能力相当于图灵机

* $|\alpha| \leq |\beta|$:G是1型文法，$L(G)$是1型语言(除 $S \rightarrow \varepsilon$);

  ---其识别系统是线性界限自动机

* $\alpha \in V$：G是2型文法，$L(G)$是2型语言;

  ---其识别系统是不确定的下推自动机

* $A \rightarrow aB$或$A \rightarrow a$：G是右线性文法，$L(G)$是3型语言

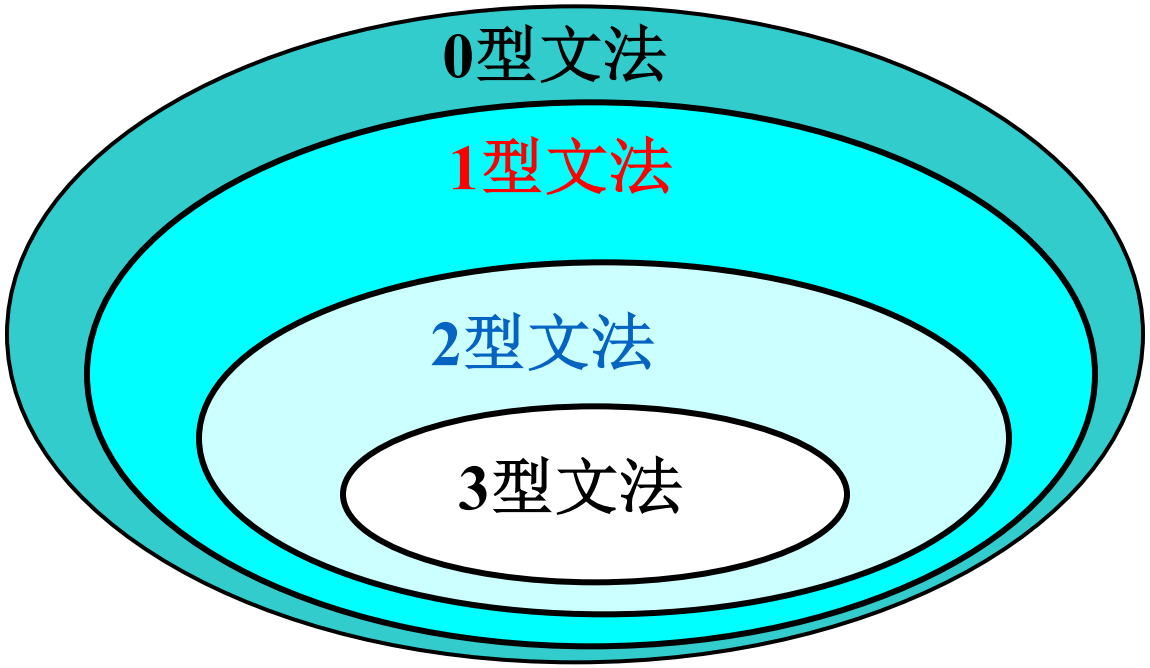
  $A \rightarrow Ba$或$A \rightarrow a$：G是左线性文法，$L(G)$是3型语言

  ---其识别系统是有穷自动机

文法的类型

四种文法之间的关系是将产生式作进一步限制而定义的。

四种文法之间的逐级“包含”关系如下：



2.5 CFG的语法树

Parse Tree

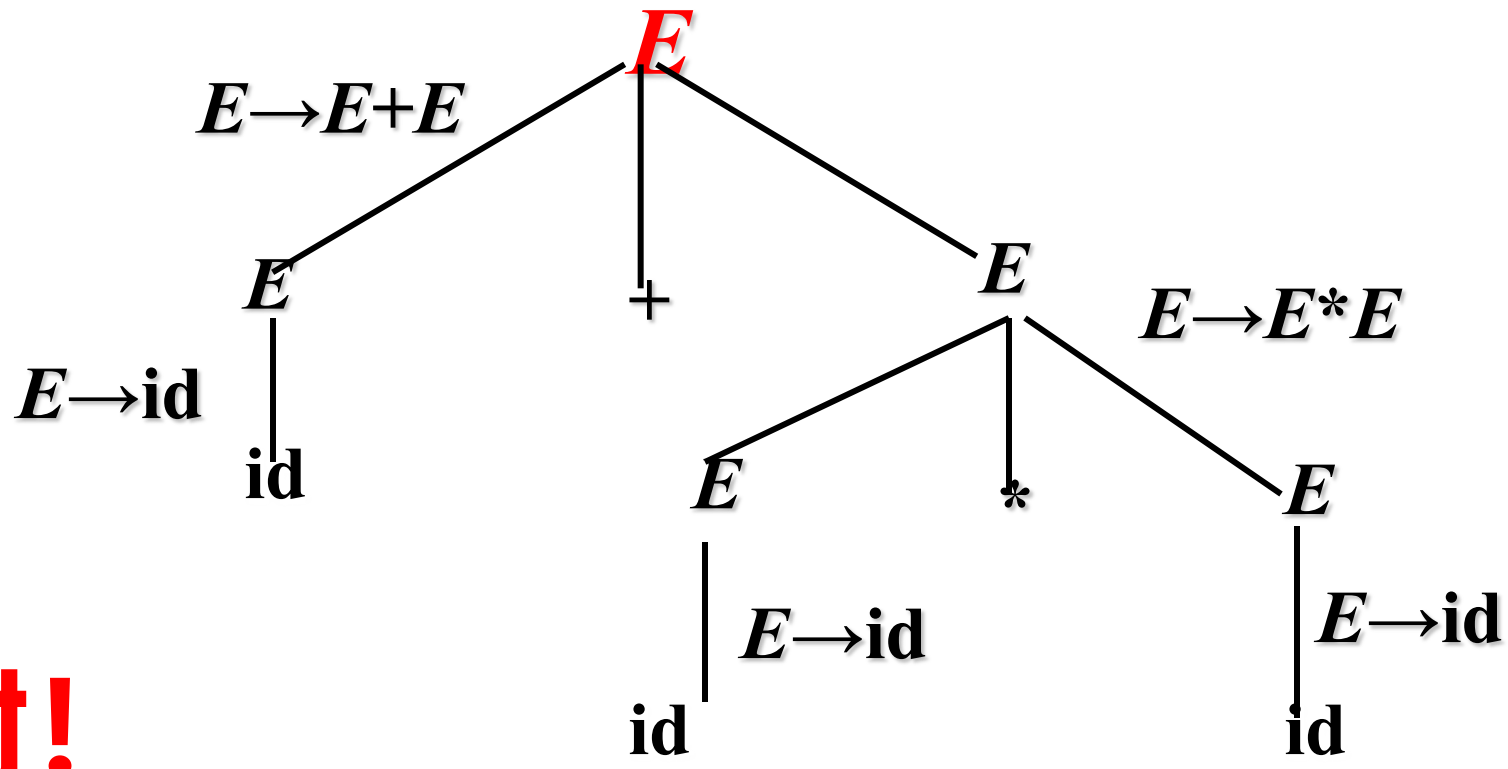
- 用树的形式表示句型的生成
  - 树根：　开始符号
  - 中间结点：　非终结符
  - 叶结点：　终结符

- 每个推导对应一个中间结点及其儿子——一个二级子树-直接短语

- 又称为分析树(parse tree)、推导树(derivation tree)、派生树(derivation tree)

例 句子结构的表示（文法$E\rightarrow E+E|E*E|(E)|\mathrm{id}$ ）

$$E\Rightarrow E+E\Rightarrow \mathrm{id}+E\Rightarrow \mathrm{id}+E*E\Rightarrow \mathrm{id}+\mathrm{id}*E\Rightarrow \mathrm{id}+\mathrm{id}*\mathrm{id}$$



一棵树！

短语(Phrase)

定义2.27 设有CFG $G=(V, T, P, S)$，$\exists \alpha, \gamma, \beta \in (V \cup T)^*$，$S \overset{*}{\Rightarrow} \gamma A \beta$，$A \overset{+}{\Rightarrow} \alpha$，则称$\alpha$是句型$\gamma\alpha\beta$的相对于变量$A$的短语(phrase)；

如果此时有$A \Rightarrow \alpha$，则称α是句型$\gamma\alpha\beta$的相对于变量A的直接短语(immediate phrase)

在无意义冲突时，简称为句型$\gamma\alpha\beta$的直接短语。直接短语也叫做简单短语(simple phrase)。

定义2.28 设有CFG $G=(V, T, P, S)$，G的句型的最左直接短语叫做句柄(handle)。

例：(直接)短语

$E \Rightarrow E+T$

$\Rightarrow T+T$

$\Rightarrow F+T$

$\Rightarrow (E)+T$

$\Rightarrow (E+T)+T$

$\Rightarrow (E+T)+T$

$\Rightarrow (T+T)+T$

$\Rightarrow (F+T)+T$

$\Rightarrow (\mathrm{id}+T)+T$

$\Rightarrow (a+T*F)+T$

$\Rightarrow (a+F*F)+T$

$\Rightarrow (a+b*F)+T$

$\Rightarrow (a+b*c)+T$

$\Rightarrow (a+b*c)+F$

$\Rightarrow (a+b*c)+d$

$E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow (E) \mid \mathrm{id}$

句柄(Handle)：最左直接短语

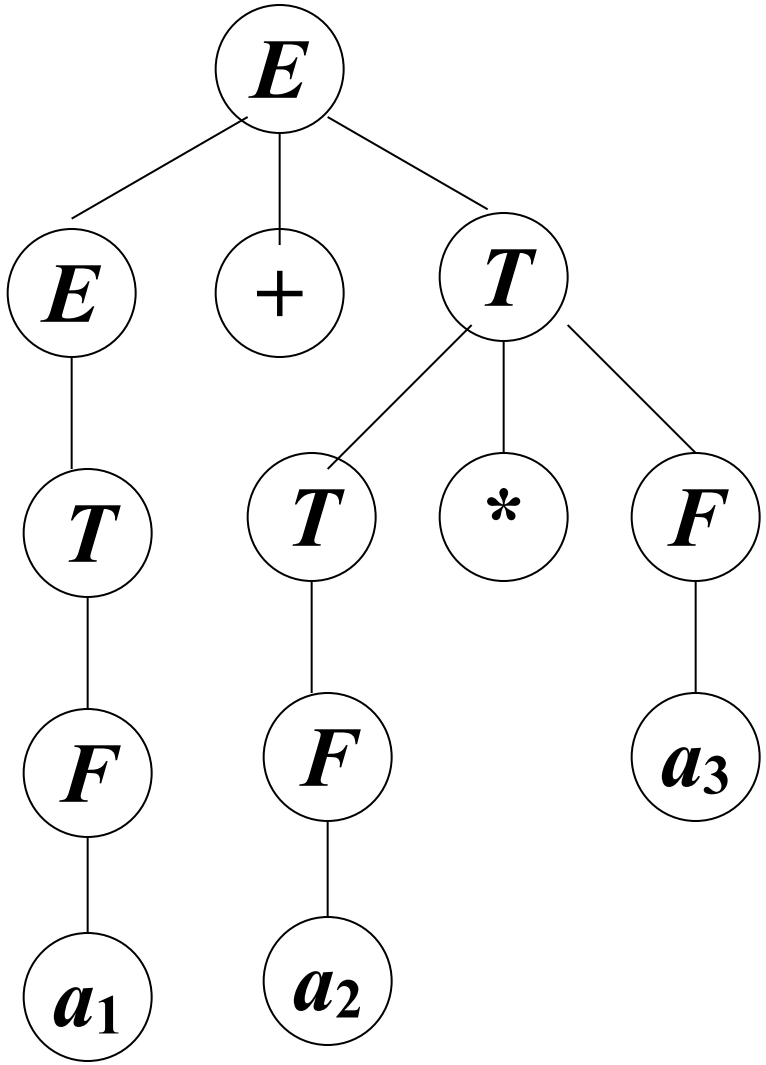
- $T \to T^* F \mid F$
- $E \to E+T \mid T$
- $F \to (E) \mid \mathrm{id}$

$$
\begin{aligned}
E &\Rightarrow E+T \\
&\Rightarrow T+T \\
&\Rightarrow F+T \\
&\Rightarrow (E)+T \\
&\Rightarrow (E+T)+T \\
&\Rightarrow (E+T)+T \\
&\Rightarrow (T+T)+T \\
&\Rightarrow (F+T)+T \\
&\Rightarrow (\mathrm{id}+T)+T \\
&\Rightarrow (a+T*F)+T \\
&\Rightarrow (a+F*F)+T \\
&\Rightarrow (a+b*F)+T \\
&\Rightarrow (a+b*c)+T \\
&\Rightarrow (a+b*c)+F \\
&\Rightarrow (a+b*c)+d
\end{aligned}
$$

用子树解释短语，直接短语，句柄

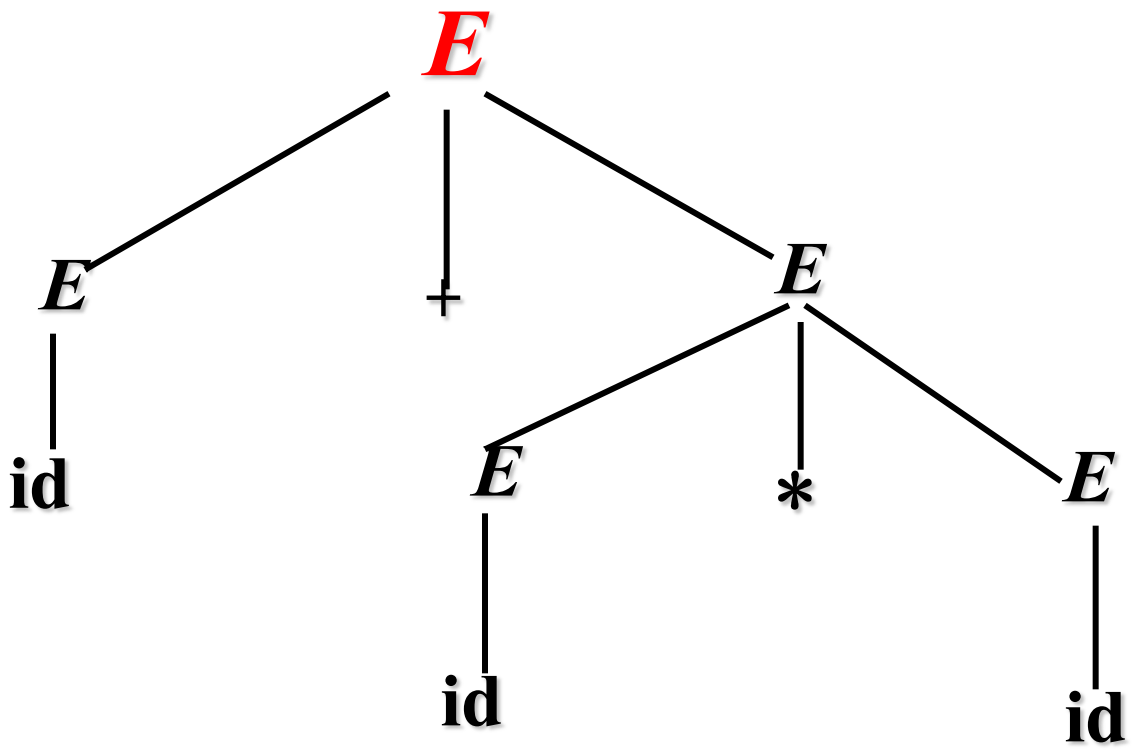
- 短语：一棵子树的所有叶子自左至右排列起来形成一个相对于子树根的短语。
- 直接短语：仅有父子两代的一棵子树，它的所有叶子自左至右排列起来所形成的符号串。
- 句柄：一个句型的分析树中最左那棵只有父子两代的子树的所有叶子的自左至右排列。
- 例如，对表达式文法$G[E]$和句子$a_1+a_2*a_3$，挑选出推导过程中产生的句型中的短语，直接短语，句柄

E	短语
$\Rightarrow \underline{E}+T$	$\underline{E}+T$
$\Rightarrow \underline{T}+T$	$\underline{T}, T+T$
$\Rightarrow \underline{F}+T$	$\underline{F}, F+T$
$\Rightarrow \underline{a_1}+T$	$\underline{a_1}, a_1+T$
$\Rightarrow \underline{a_1}+T*F$	$\underline{a_1}, \underline{T*F}, a_1+T*F$
$\Rightarrow \underline{a_1}+F*F$	$\underline{a_1}, \underline{F}, F*F,$
$\Rightarrow \underline{a_1}+a_2*F$	$\underline{a_1}, \underline{a_2}, a_1+a_2*F, a_2*F$
$\Rightarrow \underline{a_1}+a_2*a_3$	$\underline{a_1}, \underline{a_2}, \underline{a_3}, a_2*a_3$ $a_1+a_2*a_3$



例 短语与分析树

(文法$E \rightarrow E+E|E*E|(E)|\mathrm{id}$)



一棵子树的叶子！

id+id*id的不同推导$E \rightarrow E+E|E*E|(E)|id$

$E \Rightarrow E*E$

$\Rightarrow E+E*E$

$\Rightarrow E+id*E$

$\Rightarrow id+id*E$

$\Rightarrow id+id*id$

不做限制

句型 (sentential Form)

（归约）

$E \Rightarrow^{*} id+id*id$

$E \Rightarrow E+E$

$\Rightarrow id+E$

$\Rightarrow id+E*E$

$\Rightarrow id+id*E$

$\Rightarrow id+id*id$

施于最左变量

左句型(left-~)

（最右归约）

$E \Rightarrow id+id*id$

$E \Rightarrow E+E$

$\Rightarrow E+E*E$

$\Rightarrow E+E*id$

$\Rightarrow E+id*id$

$\Rightarrow id+id*id$

施于最右变量

右句型/规范句型
(canonical ~)

(最左/规范归约)

$E \Rightarrow id+id*id$

最左推导与最右推导

最左推导(Left-most Derivation)

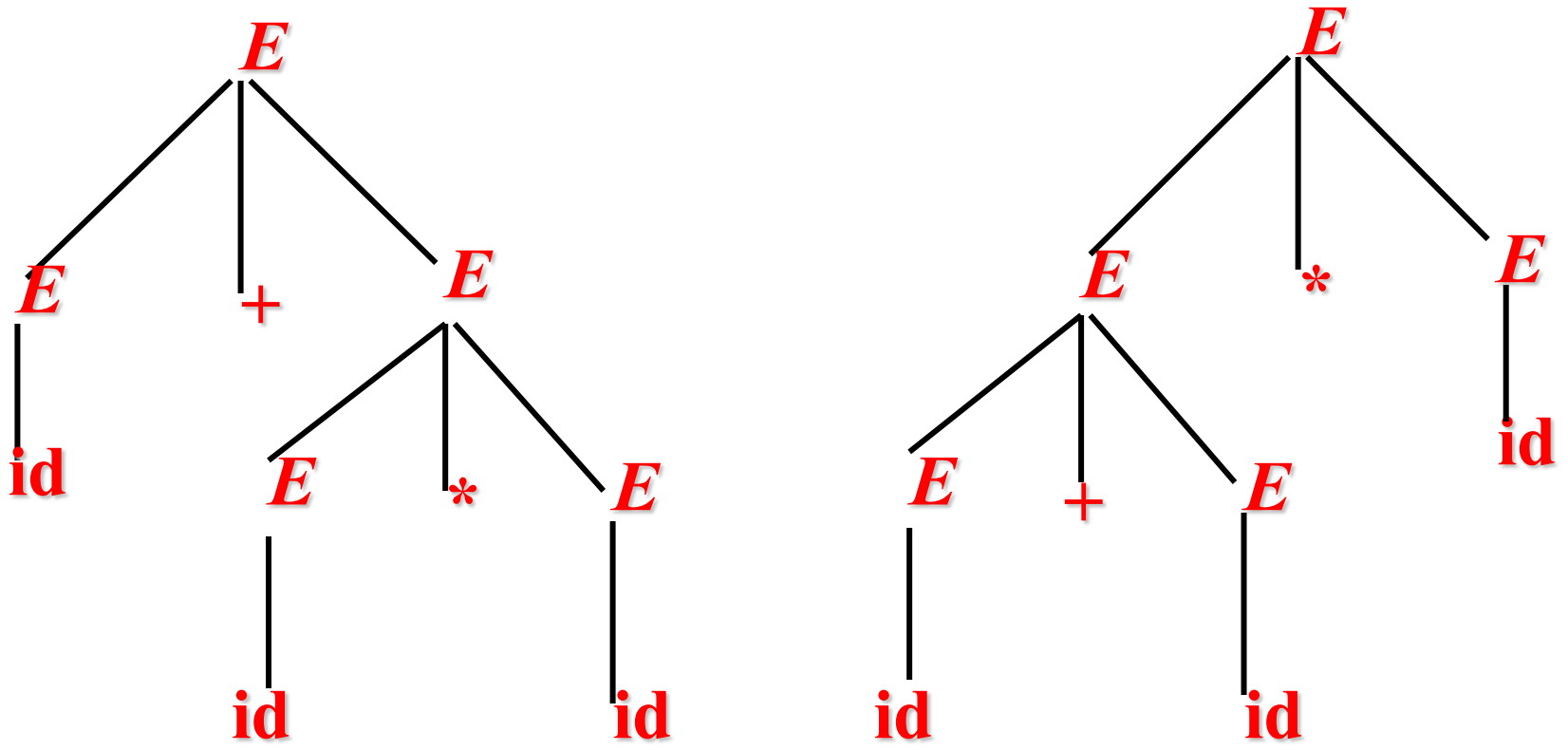
- 每次推导都施加在句型的最左边的语法变量上。——与最右归约对应

最右推导(Right-most Derivation)

- 每次推导都施加在句型的最右边的语法变量上。——与最左归约（规范规约）对应的规范(Canonical)句型

2.6 CFG的二义性

- 对同一句子存在两棵语法分析树
  - 在理论上不可判定



文法的二义性

1. 描述一个句子的文法不是唯一的;
2. 对于一个句子的分析应是唯一的。

考虑表达式下面的文法 $G[E]$,其产生式如下:

$$E \rightarrow E+E \mid E*E \mid (E) \mid a$$

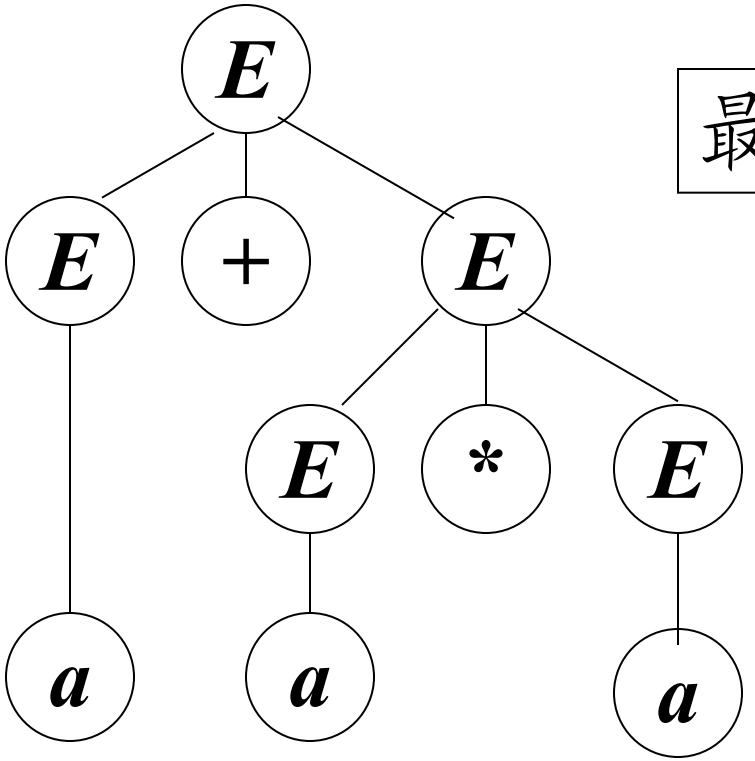
对于句子$a+a*a$, 有如下两个最左推导:

$$E \Rightarrow E+E \Rightarrow a+E \Rightarrow a+E*E \Rightarrow a+a*E \Rightarrow a+a*a$$

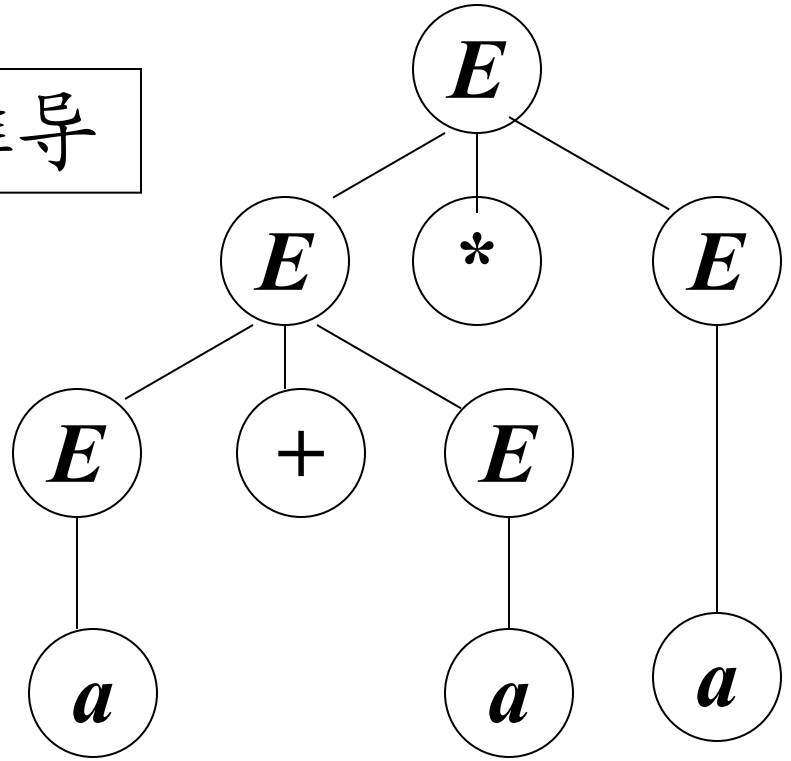
$$E \Rightarrow E*E \Rightarrow E+E*E \Rightarrow a+E*E \Rightarrow a+a*E \Rightarrow a+a*a$$

$$E \Rightarrow E+E \Rightarrow a+E \Rightarrow a+E*E \Rightarrow a+a*E \Rightarrow a+a*a$$

$$E \Rightarrow E*E \Rightarrow E+E*E \Rightarrow a+E*E \Rightarrow a+a*E \Rightarrow a+a*a$$

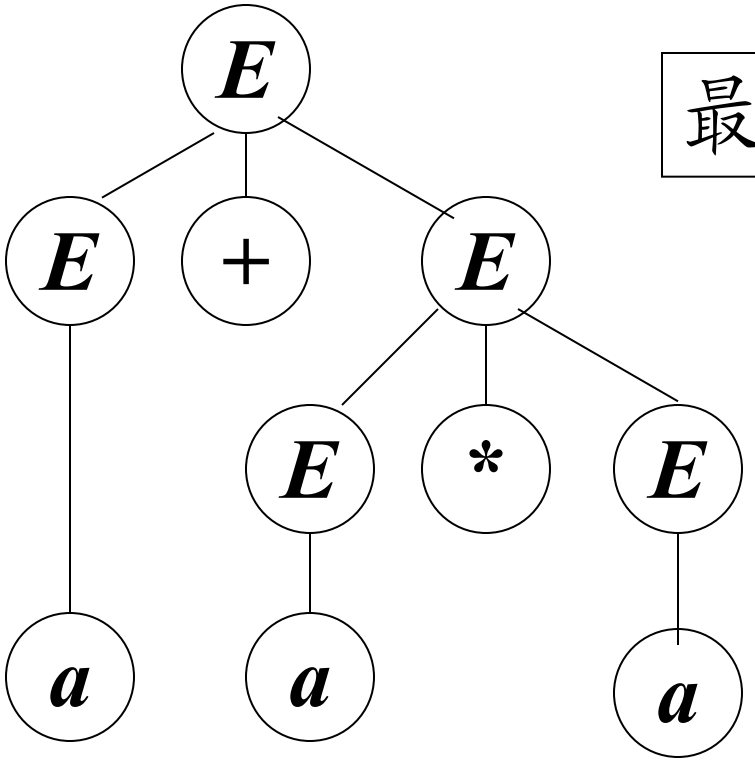


最左推导

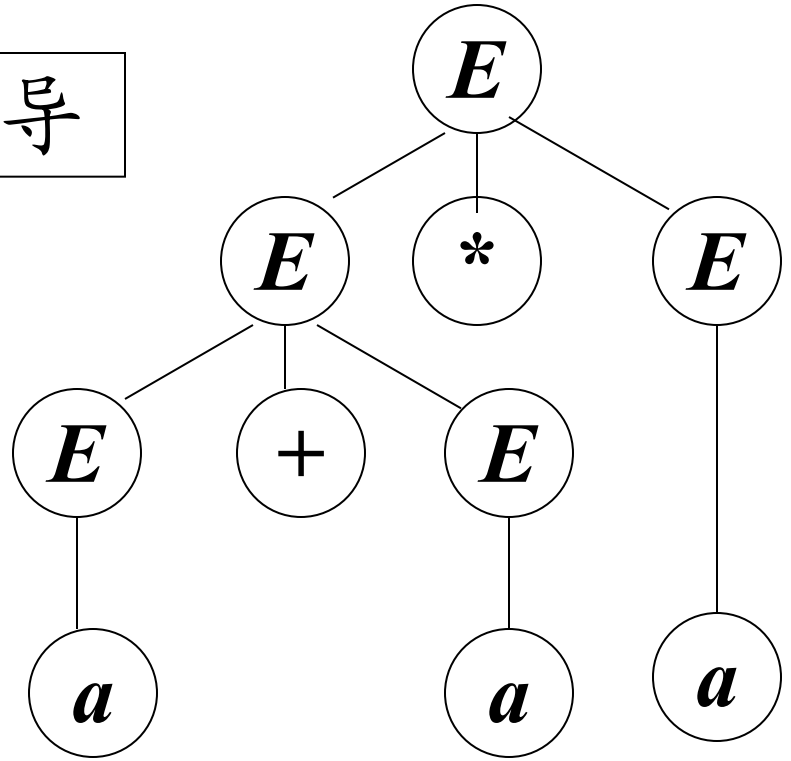


$$E \Rightarrow E+E \Rightarrow E+E*E \Rightarrow E+E*a \Rightarrow E+a*a \Rightarrow a+a*a$$

$$E \Rightarrow E*E \Rightarrow E*a \Rightarrow E+E*a \Rightarrow E+a*a \Rightarrow a+a*a$$



最右推导



二义性（ambiguity)的定义

- 如果一个文法的句子存在两棵分析树,那么,该句子是二义性的。如果一个文法包含二义性的句子,则称这个文法是二义性的; 否则，该文法是无二义性的。几点说明:
- 1 . 一般来说，程序语言存在无二义性文法， 对于表达式来说，文法（2 .1）是二义性的。对于条件语句，经常使用二义性文法描述它:
- $S \rightarrow$ if *expr* then S

  | if *expr* then S else S

  | other
- 二义性的句子: if e_1 then if e_2 then s_1 else s_2

描述if语句的无二义性文法的产生式

设：
matched_s表示所有 if 都已匹配的语句
unmatched_s 表示仍含悬空 if 的语句

下面是

$S \rightarrow matched_s \mid unmatched_s$

$matched_s \rightarrow \text{if } expr \text{ then } matched_s \text{ else } matched_s \mid \text{other}$

$unmatched_s \rightarrow \text{if } expr \text{ then } S$

$\mid \text{if } expr \text{ then } matched_s \text{ else } unmatched_s$

样例：if e1 then if e2 then s1 else s2

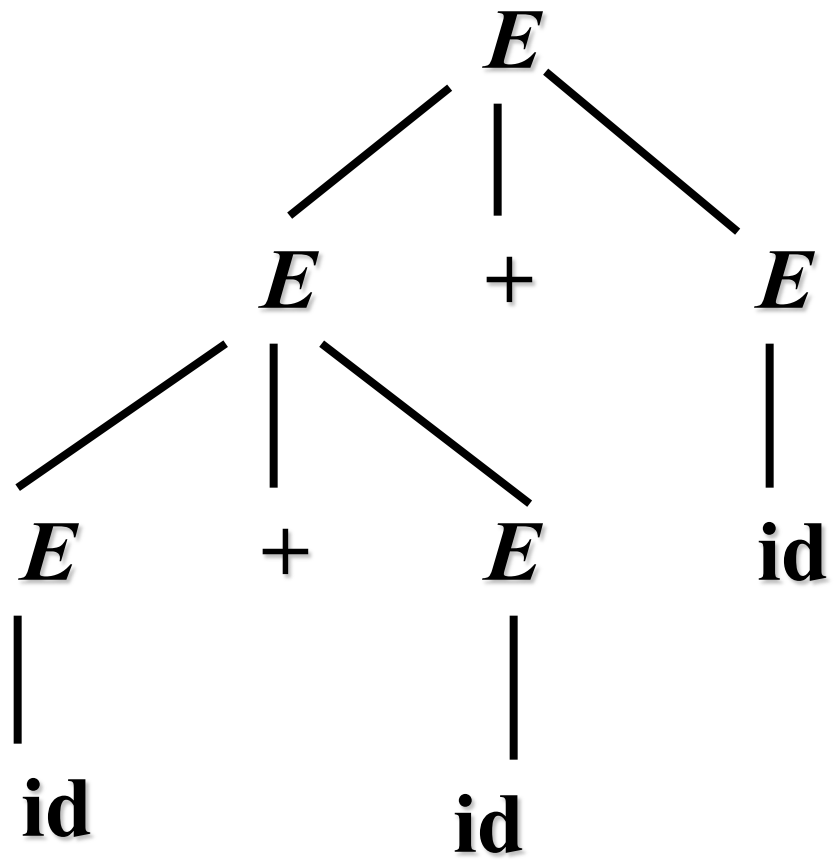
S
⇒ unmatched_s
⇒ if e1 then S
⇒ if e1 then matched_s
⇒ if e1 then if e2 then matched_s else matched_s
⇒ if e1 then if e2 then s1 else s2

该文法通过区分“已匹配语句”和“未匹配语句”，保证 else 总与最近的未匹配 if 配对，从而消除二义性。

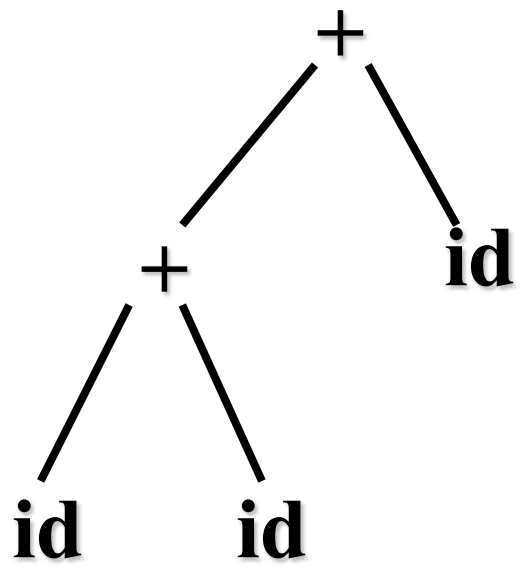
它显然比较复杂，因此：

2. 在能驾驭的情况下，使用二义性文法。

分析树与(抽象)语法树不同



(语法)分析树



(抽象)语法树

单选题 10分

设置

乔姆斯基(Chomsky)把文法分为四种类型，即0型、1型、2型、3型。其中3型文法是( )。

A 正则文法

B 上下文无关文法

C 上下文有关文法

D 非限制文法

2.7 本章小结

- 语言及其描述
- 文法的基本概念
- Chomsky体系
- CFG的语法树
- 文法的二义性
- 语言的固有二义性